

Developer Guide

River Hawk Consulting, LLC · IMAX HCD Prototype OT

August 8, 2026

CONTENTS

Developer Guide — IMAX Chat-Viewer (Phase 3 Angular build)

Architecture

Three things in here that are easy to undo by accident

Dev topology

Tests

Ground rules for new work

The OCR line contract pairs inside the object, on purpose

A bulk verdict is not a read verdict

Deployment

Repo map

Developer Guide — IMAX Chat-Viewer (Phase 3 Angular build)

UNCLASSIFIED — Deliverable ④. The Angular SPA at `angular/` is the delivery codebase; the React app at `src/` is the Phase 2 reference implementation (living spec) and ships unchanged. Testids and mock-service contracts carry between them 1:1 — that is the acceptance mechanism.

Architecture

```
angular/src/app
├── core
│   ├── models          api.models.ts (frozen wire contracts) ·
│   session.models.ts (client state)
│   ├── api             SearchApi · TranslateApi · EntitiesApi ·
│   SummarizeApi · OcrApi
│   ├── stores          TriageStore · SessionStore · GoldCopyStore
│   (signals, in-memory)
│   ├── directives      PanelResize (shared drag-to-resize for both side
│   panels)
│   └── features
│       ├── triage       TriagePanel (queue + search) · SnippetHighlight
│       ├── viewer       ChatViewer · MessageBubble · OcrDialog
│       └── gold-copy     GoldCopyPanel · ProvenanceLine
```

For the system as a whole — services, certificates, identity, deployment topology, and the values only the target environment can supply — see [architecture.md](#). That document is written to be sufficient on its own; this one assumes you are in the code.

THREE THINGS IN HERE THAT ARE EASY TO UNDO BY ACCIDENT

The message stream is virtualized (`ChatViewer`, `cdk-virtual-scroll-viewport` from `@angular/cdk/scrolling`). The fixed-size strategy is deliberate: the autosize strategy lives in `@angular/cdk-experimental`, and adding an experimental package

that the enclave's npm mirror has to carry and the vulnerability gate has to clear is a bad trade for a deliverable the Government maintains.

`itemSizePx` is **81**, and that number is measured rather than chosen: bubbles render at 67px (English) or 94px (foreign, which carries the translation line). It matters more than it looks. The value was 150 until 8 August — a guess made when the virtualization landed, which nothing in the corpus was long enough to expose. On a 2,388-message thread it made the spacer 358,000px against ~193,000px of real content, so the strategy positioned every item as though it occupied 150px, the items stacked at ~81px, and the rendered block ran out about halfway down its slot. On screen that was blank space that read as the end of the conversation. `virtual-scroll.spec.js` now measures rendered bubble heights against the reserved height and fails if a restyle moves them apart, because the enclave is the wrong place to find that out.

Two consequences worth knowing before touching that component. Anything that finds a message by `document.querySelector` will fail most of the time, because the target is usually not in the DOM — jump-to-evidence asks the viewport for the index first. And `tests/ng/virtual-scroll.spec.js` asserts a *bounded* DOM at volume; a change that quietly reverts windowing fails there rather than in the enclave. The corpus itself now carries the case: `t-3000` is a 2,388-message standing channel, held as one thread because that is how one is worked, so windowing is exercised by real demonstration data and not only by a thread synthesised inside a test.

Render order is decided in the viewer, not taken from the service. Collection feeds do not guarantee chronological delivery — the audio-cut fixtures did not — so `orderedMessages` sorts, and the promoted transcript sorts chronologically regardless of which way the linguist is reading.

Nothing is persisted. No `localStorage`, `sessionStorage`, `IndexedDB` or cookies anywhere in either build. Panel widths, sort direction, viewer geometry and all officer work live in signals and reset on reload. This is a requirement (TDD Task 3), not an oversight; `SessionStore` is the seam if production needs otherwise.

- **Angular 21 (LTS), pinned exact** in `angular/package.json`; standalone components, signals, built-in control flow. No APIs newer than v17 — the pin is a stated assumption in the kickoff correspondence and an adjacent-major `ng update` is deliberately cheap.

- **Stores** hold all client state, in-memory only (the evaluated demo's performance claim — no persistence):
 - `TriageStore` — selected thread, focus-message (with a retrigger counter so repeat hits re-flash), and per-thread dispositions. Dispositions live here, not in the panel, because promoting a thread to gold marks it worked from the viewer side.
 - `SessionStore` — cached machine translations and entities (enrichment is thread-level, run once from the workflow strip), review verdicts, analyst notes, **conversation notes keyed by threadId** (a judgement about the whole exchange, so it survives scrolling a 2,000-message stream and leads the gold copy rather than being attributed to one message), doc notes, and doc tags with the folded `tag → {threads, docs}` index that feeds the officer-tag facet.
 - `GoldCopyStore` — promoted thread-gold entries: the bench's single output (`hcd/one_output_model.md`). Snippet evidence clips were removed 1 Aug (logged deviation; single-commit revert if the Sponsor requires strict parity).
- **Enrichment is fixture-backed by default and gateway-backed by configuration.** `providers/model-gateway.js` speaks the OpenAI-compatible chat-completions wire format; supply `MODEL_ENDPOINT` and `MODEL_NAME` in the deployment ConfigMap and `MODEL_API_KEY` in the Secret (all three are environment-specific; no gateway address is committed), and translation and OCR answer from the enclave's live model instead of the seed fixtures — same contracts, no client rework. Translation sends full thread context; OCR uses vision on rasterized pages (`node scripts/rasterize-attachments.js`). Every failure path falls back to the fixture and logs. Provenance reports the engine (`<gateway-host>:<model>`), so the gold copy never misstates who translated what. Details: `deploy/README.md` §4.
- **API services** mirror the route contracts verbatim (`routes/*.js` headers are the source of truth; the contracts are frozen). Wire-format unit tests pin URL, params, and body shapes per service.
- **Escaping discipline:** corpus text renders as text nodes only — snippet highlighting builds `<mark>` spans from split text, never `innerHTML`.

Dev topology

```
npm install && npm --prefix angular install
npm run dev:ng          # mock services :5177 + ng serve :4200
(proxy.conf.json)
npm run dev             # the React reference app (:5273), same mock
services
```

Tests

```
npm --prefix angular run lint && npm --prefix angular run build && npm --
prefix angular test
npm run test:ng         # e2e vs the Angular build (Playwright boots both
servers)
npm test                # the original suite vs the React reference app
```

- `tests/*.spec.js` — the 7-spec Phase 2 acceptance suite (+ shared shell/services specs). It drives the browser by `testid`, so it runs against either app: the ported flows live verbatim inside `tests/ng/*.spec.js` and all 7 pass against the Angular build (see `project/evidence/README.md` for the coverage table).
- `tests/ng/` — Angular-build specs: the verbatim acceptance flows plus amendment coverage (queue progress, stack-clear, thread gold end-to-end). The React config ignores this directory.
- CI (`.github/workflows/angular-ci.yml`): lint → prod build → unit → e2e, plus the `package-validate` job (below) on every push.

Ground rules for new work

1. **data-testid values are the contract.** Never rename one; additions are fine (the amendment added `thread-*` / `queue-progress` / `stack-clear` testids — additive only).
2. **Mock contracts are frozen.** UI change? fine. Route shape change? that's a Sponsor conversation, not a refactor.

3. **Visual parity with the evaluated demo** — palette and layout tokens live in `styles.scss` as CSS custom properties (entity colors are part of the design contract). Parity baseline: `project/parity/`.

THE OCR LINE CONTRACT PAIRS INSIDE THE OBJECT, ON PURPOSE

`/api/ocr` returns `blocks: [{ text, en, bbox }]` — each line's English travels **on the line**, not in a parallel array. That is not a style choice. A vision model transcribing a form merges wrapped lines and splits table cells, so `lines[]` and `en[]` come back with different lengths and every row below the drift point pairs the wrong translation with the wrong source. Nothing looks broken on screen, and an officer certifies a pairing the machine never made. One object per line makes the mismatch impossible to express.

`providers/model-gateway.js` normalises whatever the gateway sends: bare strings from an older prompt still transcribe and simply carry no `en`, and the viewer falls back to the whole-document `englishGloss` rather than rendering an empty column. Pairing is all-or-nothing per page for the same reason. `tests/node/ocr-fixtures.test.js` holds the fixtures to the same guarantee, so the mock cannot promise something the enclave cannot do.

A BULK VERDICT IS NOT A READ VERDICT

`TranslationReview.bulk` marks a verdict that came from *Accept remaining* rather than from opening the message. The gold copy renders those as `bulk-accepted`, never `linguist-confirmed`. Dropping that flag would make the transcript claim an officer read text they never opened — the failure `hcd/one_output_model.md` removed the queue-side “mark reviewed” to avoid.

Deployment

The full packaging story (six images, kustomize manifests, probes/limits, statelessness, registry push, production cutover) is `deploy/README.md`. Short version:

```
./deploy/build-images.sh 0.1.0
kubectl apply -k deploy/k8s
kubectl port-forward svc/imax-spa 8443:8443
```

CI proves the set on a kind cluster every push (image builds → trivy CRITICAL gate → deploy → rollout → smoke through the SPA proxy chain). The production cutover swaps the nginx upstreams for enrichment-service endpoints — contracts frozen, no client rework.

Repo map

Path	What
<code>angular/</code>	Delivery SPA (this guide's subject)
<code>src/</code> , <code>index.html</code> , <code>vite.config.mjs</code>	React reference implementation (frozen spec)
<code>server.js</code> , <code>routes/</code> , <code>data/</code> , <code>static/</code>	Mock enrichment services + seed corpus (frozen contracts)
<code>tests/</code>	Playwright acceptance + Angular-build suites
<code>deploy/</code>	Dockerfiles, manifests, deployment guide
<code>hcd/</code>	Deliverable ① — personas, empathy maps, wireframes, workflow model
<code>docs/</code>	Deliverable ④ — this guide, the user guide, and <code>architecture.md</code> (Article XII)
<code>project/</code>	Deliverable ⑤/⑥ — risk register, burndown, evidence trail